“Compiler Design

Unit 6

Machine Independent Optimization

Available on notes.arpanadhikari7.com.np

The Principal Sources of Optimization

- A transformation of a program is called local if it can be performed by looking only at the statements in a basic block; otherwise, it is called global.
- Many transformations can be performed at both the local and global levels. Local transformations are usually performed first.

Function-Preserving Transformations

- There are a number of ways in which a compiler can improve a program without changing the function it computes.
- The transformations
  - **Common sub expression elimination,**
  - **Copy propagation,**
  - **Dead-code elimination, and**
  - **Constant folding**
- are common examples of such function-preserving transformations.

Available on notes.arpanadhikari7.com.np

- The other transformations come up primarily when global optimizations are performed.
- Frequently, a program will include several calculations of the same value, such as an offset in an array.
- Some of the duplicate calculations cannot be avoided by the programmer because they lie below the level of detail accessible within the source language.

Available on notes.arpanadhikari7.com.np

Common Sub expressions elimination:

- An occurrence of an expression E is called a common sub-expression if E was previously computed, and the values of variables in E have not changed since the previous computation.

- We can avoid re-computing the expression if we can use the previously computed value.
- For example

  t1: = 4*i

  t2: = a [t1]

  t3: = 4*j

  t4: = 4*i

  t5: = n

  t6: = b [t4] +t5

Available on notes.arpanadhikari7.com.np

- The above code can be optimized using the common sub-expression elimination as

  t1: = 4*i

  t2: = a [t1]

  t3: = 4*j

  t5: = n

  t6: = b [t1] +t5

- The common sub expression t4: =4*i is eliminated as its computation is already in t1.

- And value of i is not been changed from definition to use

Available on notes.arpanadhikari7.com.np

Copy Propagation:

- Assignments of the form f : = g called copy statements, or copies for short.
- The idea behind the copy-propagation transformation is to use g for f, whenever possible after the copy statement f: = g.
- Copy propagation means use of one variable instead of another.
- This may not appear to be an improvement, but as we shall see it gives us an opportunity to eliminate x.
- For example:

  x=Pi;

  A=x*r*r;
- The optimization using copy propagation can be done as follows:

  A=Pi*r*r;
- Here the variable x is eliminated

Available on notes.arpanadhikari7.com.np

Dead-Code Eliminations:

- A variable is live at a point in a program if its value can be used subsequently; otherwise, it is dead at that point.
- A related idea is dead or useless code, statements that compute values that never get used.
- While the programmer is unlikely to introduce any dead code intentionally, it may appear as the result of previous transformations.
- An optimization can be done by eliminating dead code.
- Example:

```
i=0;
if(i=1)
{
a=b+5;
}
```

- Here, „if“ statement is dead code because this condition will never get satisfied.

Available on notes.arpanadhikari7.com.np

Constant folding:

- We can eliminate both the test and printing from the object code.
- More generally, deducing at compile time that the value of an expression is a constant and using the constant instead is known as constant folding.
- One advantage of copy propagation is that it often turns the copy statement into dead code.
- For example,

  a=3.14157/2 can be replaced by

  a=1.570 there by eliminating a division operation.

Available on notes.arpanadhikari7.com.np

Loop Optimizations:

- We now give a brief introduction to a very important place for optimizations, namely loops, especially the inner loops where programs tend to spend the bulk of their time.
- The running time of a program may be improved if we decrease the number of instructions in an inner loop, even if we increase the amount of code outside that loop.
- Three techniques are important for loop optimization:
- **Code motion**, which moves code outside a loop;
- **Induction-variable elimination**, which we apply to replace variables from inner loop.
- **Reduction in strength**, which replaces and expensive operation by a cheaper one, such as a multiplication by an addition.

Available on notes.arpanadhikari7.com.np

Code Motion:

- An important modification that decreases the amount of code in a loop is code motion.
- This transformation takes an expression that yields the same result independent of the number of times a loop is executed (a loop-invariant computation) and places the expression before the loop.
- Note that the notion "before the loop" assumes the existence of an entry for the loop.
- For example, evaluation of limit-2 is a loop-invariant computation in the following while-statement:
- while (i <= limit-2) /* statement does not change limit*/
- Code motion will result in the equivalent of

  t= limit-2;

  while (i<=t) /* statement does not change limit or t*/

Available on notes.arpanadhikari7.com.np

Reduction In Strength:

- Reduction in strength replaces expensive operations by equivalent cheaper ones on the target machine.
- Certain machine instructions are considerably cheaper than others and can often be used as special cases of more expensive operators.
- For example, x^2 is invariably cheaper to implement as x*x than as a call to an exponentiation routine.
- Fixed-point multiplication or division by a power of two is cheaper to implement as a shift.
- Floating-point division by a constant can be implemented as multiplication by a constant, which may be cheaper.

Available on notes.arpanadhikari7.com.np

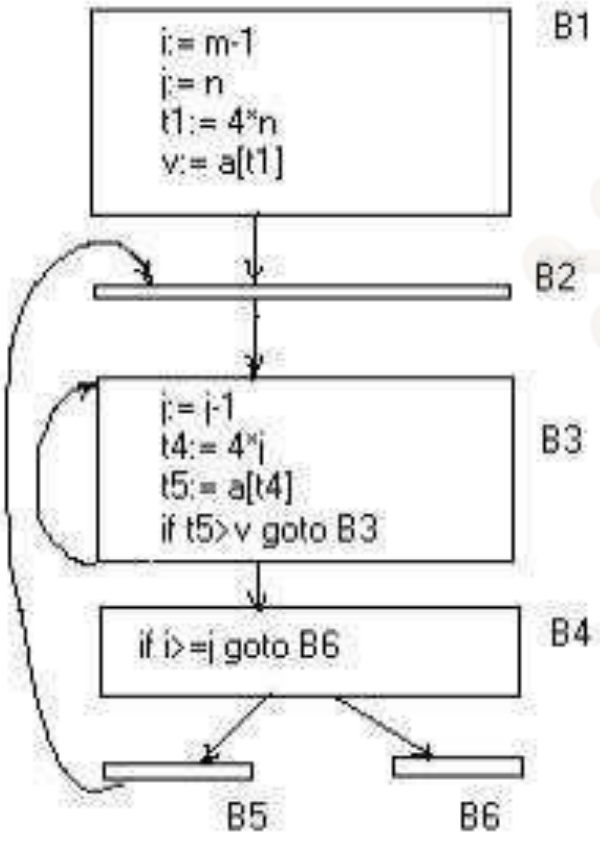
Induction Variables :

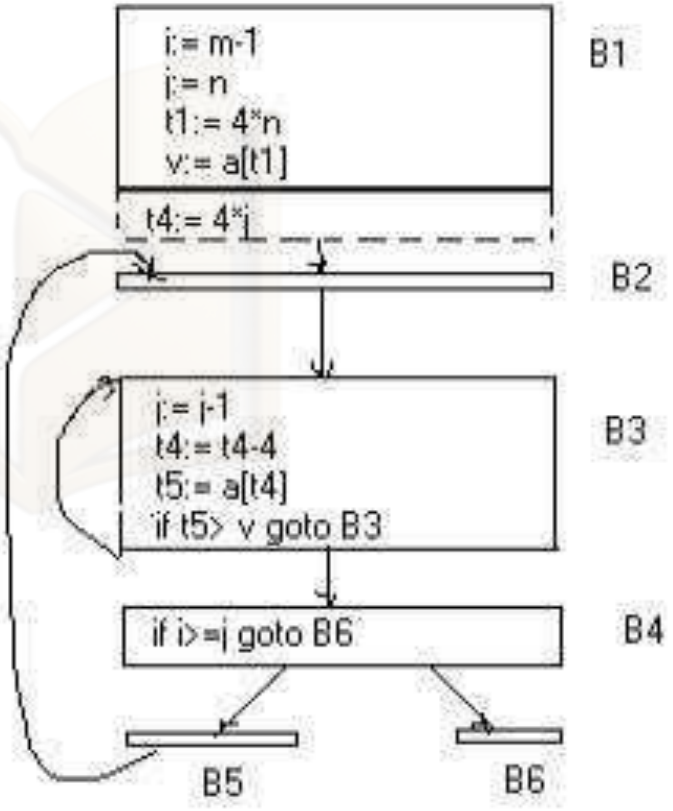
- Loops are usually processed inside out. For example consider the loop around B3.
- The values of j and t 4 remain in lock-step; every time the value of j decreases by 1, that of t4 decreases by 4 because 4*j is assigned to t4.
- Such identifiers are called induction variables.
- When there are two or more induction variables in a loop, it may be possible to get rid of all but one, by the process of induction-variable elimination.
- For the inner loop around B3 in Fig. we cannot get rid of either j or t4 completely; t4 is used in B3 and j in B4.
- However, we can illustrate reduction in strength and illustrate a part of the process of induction-variable elimination.
- Eventually j will be eliminated when the outer loop of B2 - B5 is considered

Available on notes.arpanadhikari7.com.np

- Example:
- As the relationship t4:=4*j surely holds after such an assignment to t4 in Fig. and t4 is not changed elsewhere in the inner loop around B3, it follows that just after the statement j:=j-1 the relationship t4:= 4*j-4 must hold.
- We may therefore replace the assignment t4:= 4*j by t4:= t4-4.
- The only problem is that t4 does not have a value when we enter block B3 for the first time.
- Since we must maintain the relationship t4=4*j on entry to the block B3, we place an initializations of t4 at the end of the block where j itself is initialized, shown by the dashed addition to block B1 in second Fig

Available on notes.arpanadhikari7.com.np





Available on notes.arpanadhikari7.com.np

- The replacement of a multiplication by a subtraction will speed up the object code if multiplication takes more time than addition or subtraction

Available on notes.arpanadhikari7.com.np

Loops in flow graph

- A graph representation of three-address statements, called a flow graph, is useful for understanding code-generation algorithms, even if the graph is not explicitly constructed by a code-generation algorithm.
- Nodes in the flow graph represent computations, and the edges represent the flow of control.

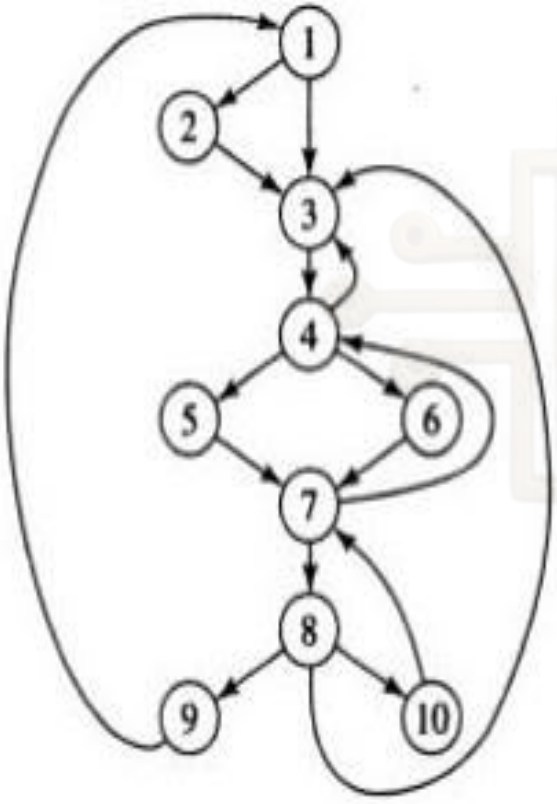
Dominators:

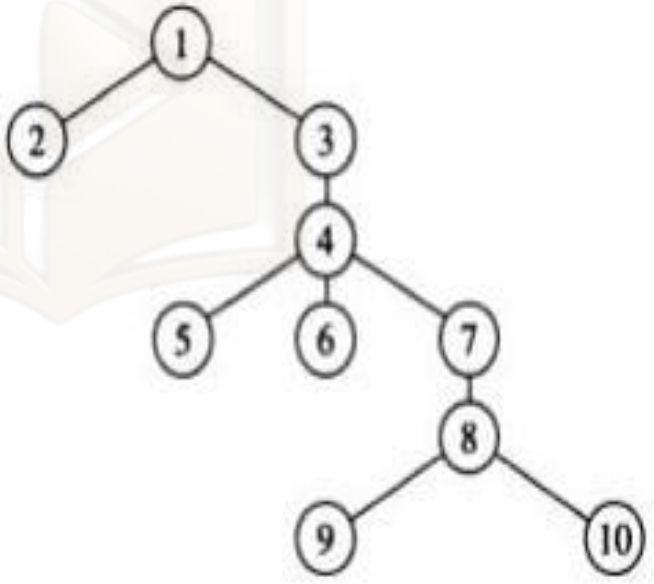
- In a flow graph, a node d dominates node n, if every path from initial node of the flow graph to n goes through d.
- This will be denoted by d dom n.
- Every initial node dominates all the remaining nodes in the flow graph and the entry of a loop dominates all nodes in the loop. Similarly every node dominates itself.

Available on notes.arpanadhikari7.com.np

- Example:
- In the flow graph below,
- Initial node,node1 dominates every node. *node 2 dominates itself *node 3 dominates all but 1 and 2. *node 4 dominates all but 1,2 and 3. *node 5 and 6 dominates only themselves, since flow of control can skip around either by going through the other. node 7 dominates 7,8 ,9 and 10. *node 8 dominates 8,9 and 10. *node 9 and 10 dominates only themselves.

Available on notes.arpanadhikari7.com.np





Available on notes.arpanadhikari7.com.np

- The way of presenting dominator information is in a tree, called the dominator tree, in which
  - The initial node is the root.
  - The parent of each other node is its immediate dominator.
  - Each node d dominates only its descendents in the tree.
- The existence of dominator tree follows from a property of dominators; each node has a unique immediate dominator in that is the last dominator of n on any path from the initial node to n.
- In terms of the dom relation, the immediate dominator m has the property is d=!n and d dom n, then d dom m.

Available on notes.arpanadhikari7.com.np

D(1)={1}

D(2)={1,2}

D(3)={1,3}

D(4)={1,3,4}

D(5)={1,3,4,5}

D(6)={1,3,4,6}

D(7)={1,3,4,7}

D(8)={1,3,4,7,8}

D(9)={1,3,4,7,8,9}

D(10)={1,3,4,7,8,10}

Available on notes.arpanadhikari7.com.np

Natural Loops:

- One application of dominator information is in determining the loops of a flow graph suitable for improvement.
- There are two essential properties of loops:
- Ø A loop must have a single entry point, called the header. This entry point-dominates all nodes in the loop, or it would not be the sole entry to the loop.
- Ø There must be at least one way to iterate the loop(i.e.)at least one path back to the header
- One way to find all the loops in a flow graph is to search for edges in the flow graph whose heads dominate their tails. If a→b is an edge, b is the head and a is the tail.
- These types of edges are called as back edges.

Available on notes.arpanadhikari7.com.np

Example:

- In the above graph,
- 7→4 4 DOM 7
- 10 →7 7 DOM 10
- 4→3
- 8→3
- 9 →1
- The above edges will form loop in flow graph. Given a back edge n → d, we define the natural loop of the edge to be d plus the set of nodes that can reach n without going through d. Node d is the header of the loop.

Available on notes.arpanadhikari7.com.np

Inner loops:

- If we use the natural loops as "the loops", then we have the useful property that unless two loops have the same header, they are either disjointed or one is entirely contained in the other.

- Thus, neglecting loops with the same header for the moment, we have a natural notion of inner loop: one that contains no other loop.

- When two natural loops have the same header, but neither is nested within the other, they are combined and treated as a single loop.

Available on notes.arpanadhikari7.com.np

Pre-Headers:

- Several transformations require us to move statements "before the header".
- Therefore begin treatment of a loop L by creating a new block, called the preheader.
- The pre-header has only the header as successor, and all edges which formerly entered the header of L from outside L instead enter the pre-header.
- Edges from inside loop L to the header are not changed.
- Initially the pre-header is empty, but transformations on L may place statements in it.

Available on notes.arpanadhikari7.com.np

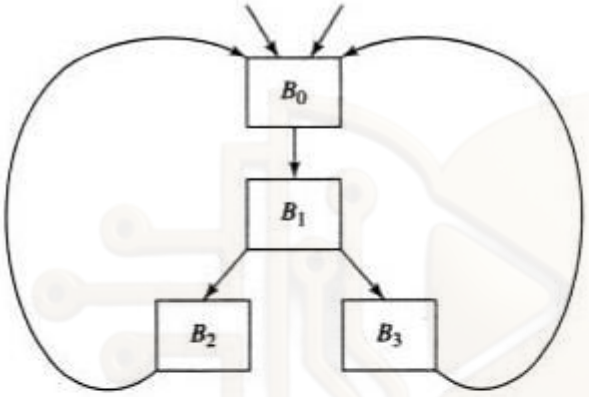


Fig. 5.4 Two loops with the same header

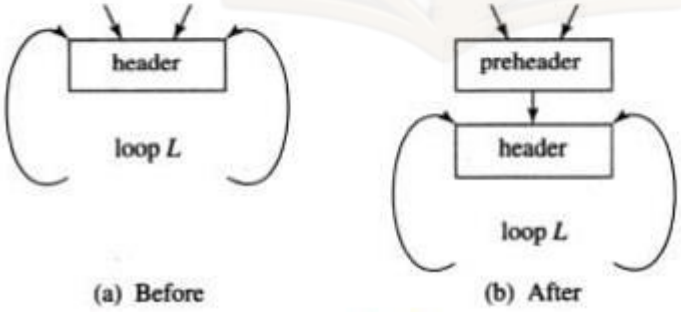


Fig. 5.5 Introduction of the preheader

Available on notes.arpanadhikari7.com.np

Reducible flow graphs:

- Reducible flow graphs are special flow graphs, for which several code optimization transformations are especially easy to perform, loops are unambiguously defined, dominators can be easily calculated, data flow analysis problems can also be solved efficiently.
- Exclusive use of structured flow-of-control statements such as if-then-else, while-do, continue, and break statements produces programs whose flow graphs are always reducible.
- The most important properties of reducible flow graphs are that
- 1. There are no umps into the middle of loops from outside;
- 2. The only entry to a loop is through its header

Available on notes.arpanadhikari7.com.np

Definition:

- A flow graph G is reducible if and only if we can partition the edges into two disjoint groups, forward edges and back edges, with the following properties.
- 1. The forward edges from an acyclic graph in which every node can be reached from initial node of G.
- 2. The back edges consist only of edges where heads dominate theirs tails.
- Example: The above flow graph is reducible. If we know the relation DOM for a flow graph, we can find and remove all the back edges. The remaining edges are forward edges. If the forward edges form an acyclic graph, then we can say the flow graph reducible. In the above example remove the five back edges 4→3, 7→4, 8→3, 9→1 and 10→7 whose heads dominate their tails, the remaining graph is acyclic.

Available on notes.arpanadhikari7.com.np

Partial Redundancy Elimination

- Redundancy is basically unnecessary or repetitive code present in a program that does not contribute to the functionality of the program.
- When a computation is performed more than once along some of the execution paths but not along all execution paths, this condition is known as "partial redundancy."
- An example of full redundancy is given below.

```
if(condition 1)
{
 x = a + b;
}
 else
{
 x = a + b;
}
```

Available on notes.arpanadhikari7.com.np

- The computation of A+B is fully redundant, as it is performed along both execution paths.
- But the condition of partial redundancy arrived if we modified the code as follows.

```
if(condition 1)
{
 x = a + b;
}
if(condition 2)
{
y = a + b;
}
```

- A+B is only partially redundant because it is performed along some execution path("given that both conditions are true") but not along all execution paths.

Available on notes.arpanadhikari7.com.np

The Source of Redundancy

- Redundancy is due to the presence of one or multiple expressions from the following expressions.
- **Partially redundant expressions:** It is an expression in which the value that is computed by the expression is already available on some of the paths but is not available on all the paths through a program to that expression.

```
if (condition)
{
   x = a + b;  // Computes (a + b)
}
y = a + b;      // Redundant if condition was true, not redundant otherwise.
```

Available on notes.arpanadhikari7.com.np

- **Loop invariant expression:** It is a condition that is necessarily true immediately before and immediately after each iteration of the loop.

```
for (int i = 0; i < n; i++) {
    x = y * z;  // (y * z) is loop-invariant if y and z don't change in
      the loop.
}
```

- **Common sub-expression:** A common sub-expression is an expression that has appeared and computed before and appears again during the computation of the code.

```
a = x + y * z;
b = w + y * z;  // (y * z) is a common sub-expression.
```

Available on notes.arpanadhikari7.com.np

Can all Redundancies be Eliminated?

- One of the questions that might arise is whether it is possible to remove all the redundant computations along every path.
- The answer depends on whether we are allowed to alter the flow graph by creating new blocks.
- All redundancies can be eliminated if we are allowed to change the flow graph.
- While if we are not permitted to change the flow graph, then all the redundancies cannot be eliminated.

Available on notes.arpanadhikari7.com.np

If We Can Modify the CFG (Add New Blocks):

- All redundancies can be eliminated.

- Partial Redundancy Elimination (PRE) can be extended to fully redundant cases by inserting computations in new blocks at merge points.

```
if (condition) {
   x = a + b;  // (a + b) computed here
} else {
   // (a + b) not computed here
}
y = a + b;     // Partially redundant
```

  - Insert a new block before the merge point to compute (a + b) once, making it fully redundant.

Available on notes.arpanadhikari7.com.np

If We Cannot Modify the CFG (No New Blocks Allowed):

- Not all redundancies can be eliminated.
- Some redundancies remain because we cannot safely insert computations without altering the program structure.

```
if (condition) {
    x = a + b;  // (a + b) computed here
}
y = a + b;      // Still partially redundant (no safe place to insert without
                   CFG changes)
```

- Without adding a new block, we cannot guarantee (a + b) is computed on all paths before y = a + b.

Available on notes.arpanadhikari7.com.np

The Lazy Code Motion Algorithm (LCM)

- The Lazy code motion algorithm (LCM) is used for the purpose of PRE, which aims to minimize the total number of computations that is to be performed to execute a program.

- **Minimizes redundant computations** by moving expressions to the **earliest safe point** in the control flow graph (CFG).

- **Preserves program semantics** (does not change the program's behavior).

- **Ensures computations are performed at most once** (no unnecessary re-executions).

Available on notes.arpanadhikari7.com.np

It is a Four-Step Algorithm:

- **Step 1:** The first step uses anticipation to determine where the expression can be placed.

- **Step 2**: The second step will be computing the earliest and latest points for each expression.

- **Step 3**: The third step then pushes the cut set down to the point where any further delay would alter the semantics of the program and may even introduce redundancy in the program.

- **Step 4:** The final step will be to clean up the pod, and we can do that by removing the assignment to temporary variables that were used previously.

- Each step should be accomplished with a data flow. The first and fourth are backward flow programs. At the same time, the 2nd and the 3rd are forward flow problems.

Available on notes.arpanadhikari7.com.np

Constant Propagation

- **Constant Propagation** is a compiler optimization technique that identifies variables or expressions that have constant values and replaces their uses with the actual constants.
- This eliminates unnecessary computations and enables further optimizations.
- Constant propagation involves analyzing code to find expressions in which the operands are constants and then replacing variables with their constant values wherever possible.

Available on notes.arpanadhikari7.com.np

Benefits:

- **Improved Performance**: Fewer runtime computations.
- **Simpler Code**: Can enable further optimizations like constant folding and dead code elimination.
- **Reduced Code Size**: Unnecessary assignments or calculations can be removed.

Available on notes.arpanadhikari7.com.np

Before constant propagation:

```
int x = 5;
int y = x + 3;
int z = y * 2;
```

After constant propagation:

```
int x = 5;
int y = 8;
int z = 16;
```

Available on notes.arpanadhikari7.com.np

Local Constant Propagation

Optimizes within a single basic block.

Example:

```
int x = 5;       // x is a constant 5
int y = x + 2;   // Replaced with y = 7
```

Global Constant Propagation

Works across multiple blocks in a control flow graph (CFG).

Example:

```
int x = 10;
if (condition) {
   y = x + 1;    // y = 11
} else {
   z = x * 2;    // z = 20
}
```

Available on notes.arpanadhikari7.com.np

Introduction to Data-Flow Analysis

- To efficiently optimize the code compiler collects all the information about the program and distribute this information to each block of the flow graph.
- This process is known as dataflow graph analysis.
- Data flow analysis is a fundamental technique used in compiler design to gather information about how data flows through a program.
- Certain optimization can only be achieved by examining the entire program.
- It can't be achieve by examining just a portion of the program.
- In order to find out the value of any variable (A) of a program at any point (P) of the program, Global data flow analysis is required.

Available on notes.arpanadhikari7.com.np

use-definition (or ud-) chaining

- Use-Definition (UD) chaining is a fundamental data flow analysis technique that identifies the relationship between variable uses and their corresponding definitions in a program.
- **Definition (Def)**: A statement that assigns a value to a variable (e.g., x = 5)
- **Use**: A statement that references a variable's value (e.g., y = x + 3)
- A UD-chain for a variable use is the set of all definitions of that variable that can reach that use along some path in the control flow graph.
- **Example**

```
x = 5
y = 10
if (z > 0)
  x = 20
print(x + y)
```

- The UD-chains would be:
- Use of x at line 5: {definition at line 1, definition at line 4}
- Use of y at line 5: {definition at line 2}

Available on notes.arpanadhikari7.com.np

- Given that variable A is used at point p, at which points could the value of A used at p have been defined?
- By a use of variable A means any occurrence of A as an operand.
- By a definition of A means either an assignment to A or the reading of a value for A.
- By a point in a program means the position before or after any intermediate language statement.
  - Control reaches a point just before a statement when that statement is about to be executed.
  - Control reaches a point after a statement when that statement has just been executed.

Available on notes.arpanadhikari7.com.np

- A definition of a variable A reaches a point p ,if there is a path in the flow graph from that definition to p, such that no other definitions of A appear on the path.
- To determine the definitions that can reach a given point in a program requires assigning distinct number to each definition.
- To compute two sets for each basic block B:
  - GEN[B]-set of generated definitions within block B.
  - KILL[B]- set of definitions outside of B that define identifiers that also have definition within B.
- To compute the sets IN[B] and OUT[B]:
- **IN[B]-**set of all definition reaching the point just before the first statement of block B.
- **OUT[B]**-set of definitions reaching the point just after the last statement of B

Available on notes.arpanadhikari7.com.np

Solving Data-Flow Equation

- **Solving data flow equations** is central to data flow analysis (DFA). It involves computing information (like reaching definitions, live variables, etc.) at each program point by solving **flow equations** over a **control flow graph (CFG)** until the solution **reaches a fixed point** (i.e., no further changes).

- The **iterative solution** is the most common method to solve **data flow equations** in a compiler.

- It repeatedly evaluates the data flow equations over the **control flow graph (CFG)** until a **fixed point** is reached — i.e., when the IN and OUT sets no longer change.

Available on notes.arpanadhikari7.com.np

- The equations for many data flow problems are similar in form in that information is being generated and killed .
- However there are two basic principal ways in which equations differ in detail.
- The equations for reaching definitions are forward equations in the sense that the out sets are computed in terms of the in sets. We shall also see data flow problems that are backward in that the in sets are computed from the out sets.

Available on notes.arpanadhikari7.com.np

- When there is more than one edge entering block B, the definition reaching the beginning of B are the union of the definitions arriving along each of the edge. We therefore say that union is the confluence operation . In contrast we shall consider problems like global available expressions, where union is the confluence operator because an expression is only available at the beginning of B if it is available at the end of every predecessor of B.

Available on notes.arpanadhikari7.com.np

Iterative algorithm for reaching definition

- A **definition** of a variable (e.g., x = ...) **reaches** a point in the program if there exists a path from that definition to the point **without any redefinition** of that variable along the path.
- Let B be a basic block
- GEN[B] = definitions generated (locally) in B
- KILL[B] = definitions **killed** (overwritten) by B
- IN[B] = set of definitions reaching the **start** of B
- OUT[B] = set of definitions reaching the **end** of B

Available on notes.arpanadhikari7.com.np

Input: Control Flow Graph (CFG), GEN and KILL sets for all blocks

1. For all basic blocks B:
   IN[B] ← ∅
   OUT[B] ← GEN[B] // Initialize OUT with GEN

2. Repeat
   For each basic block B:
      IN[B] ← ∪ OUT[P] // over all predecessors P of B
      OUT[B] ← GEN[B] ∪ (IN[B] - KILL[B])
  Until OUT sets do not change for any block (fixed point)

Available on notes.arpanadhikari7.com.np

- The algorithm works by **propagating definitions** forward through the CFG.
- **IN[B]** gathers definitions from all predecessor blocks.
- **OUT[B]** is formed by:
- Keeping the new definitions from GEN[B]
- And keeping only the previous definitions that are **not killed** by KILL[B]

Available on notes.arpanadhikari7.com.np

Data Flow Problems and Their Equations

Problem	Direction	Data Flow Equation(s)
Reaching Definitions	Forward	IN[B] = ∪ OUT[P] OUT[B] = GEN[B] ∪ (IN[B] - KILL[B])
Live Variable Analysis	Backward	OUT[B] = ∪ IN[S] IN[B] = GEN[B] ∪ (OUT[B] - KILL[B])
Available Expressions	Forward	IN[B] = ∩ OUT[P] OUT[B] = GEN[B] ∪ (IN[B] - KILL[B])
Very Busy Expressions	Backward	IN[B] =∩ OUT[P] OUT[B] = GEN[B] ∪ (OUT[B] - KILL[B])

Available on notes.arpanadhikari7.com.np